

Guardrails and governance for viax surfaces.

Forma is the umbrella name for the way AI composes viax applications: a coordinating skill maps schema to UXM components, a fixed set of skills carry the cross-cutting concerns no portal should reinvent, and every build emits a `design.md` so humans review the spec before code lands. The first proof is Modo. The second is the generic Revenue Motion portal.

Problem & Target

TODAY

AI can produce any UI you ask for. **That is the problem.** Without a frame, every portal is a one-off — bespoke forms, inconsistent components, role visibility bolted on, localization deferred, security reasoned about per screen. Reuse is theatre. Each engagement starts the work over.

TARGET

AI builds **with** Forma — composing UXM components against schema, behind a fixed set of skills that handle the things no portal should reinvent. Every build emits a `design.md` — the human-reviewable contract for what got composed. **Engagements compound.** Customers extend on the same surfaces viax delivers on, *in the JS framework of their choice.*

The Stack

SURFACE

Modo

Home for viax modeling functionality. Shipped by viax, built with UXM components.

SURFACE

Portals

The generic Revenue Motion portal and custom customer surfaces. Composed by Forma (post-config); client picks the JS framework.

▼ RESTS ON

PATTERN

Forma

The umbrella for how AI composes viax applications. A coordinating skill maps schema to UXM components; fixed skills carry the cross-cutting concerns AI *must never* write per screen.

- Schema → component mapping (coordinating skill)
- Dynamic forms
- Role-based security & visibility
- Localization & i18n
- Multi-framework output (Vue today; React and others ahead)

▼ COMPOSES FROM

EXPERIENCE

UXM

The core viax component package. **Schema-agnostic** — components take props, emit events. Brand identity, accessibility, and variant consistency live *here* — not in code AI generates per portal.

- 67 components (App · Buttons · Inputs · ...)
- Schema-agnostic (props in, events out)
- Brand Settings (logo, type, light/dark)
- WCAG AA enforced; variants stay consistent
- Customers customize tokens & add components; core stays viax-owned

Every Forma build emits a `design.md` — the **human-reviewable spec** for what got composed. AI is judged on the spec before code lands.

Why this matters for delivery

Each engagement **compounds** instead of resetting. Customers extend on the same surfaces viax delivers on — there is no fork — and choose the JS framework that fits their stack. AI gets *faster and less risky* over time, because every new capability lands inside Forma's skills or UXM's library, not on a blank file. **Modo is the proof:** viax ships its own product on the same components we ask customers to compose theirs from.

Principles

NON-NEGOTIABLES

- Things are built **with** Forma, never **on** it.
- Skills **replace bespoke plumbing** — forms, security, visibility, i18n.
- Customers **customize tokens and add components**; core components stay viax-owned.
- Every Forma build emits a `design.md` — the **spec is the artifact** AI is judged on, before code lands.
- UXM is the only place components are defined; Forma **composes from** it, never around it.
- Brand identity, accessibility, and variant consistency are **UXM's job** — never AI's per-screen decision.
- Customers **pick the JS framework** for their surfaces; the component contract and skills are not.